Pub Crawl is coming back, March 11–12! Register Now!

Member-only story

BPC > Try for full article text: | freedium.cfd

The Container Orchestration War That Kubernetes Is Losing

5 min read · Oct 11, 2025

Sohail Saifi Follow

Listen Share More

Open in app ↗

Medium

Kubernetes won the container orchestration war, right? That's what everyone keeps saying. It's the industry standard. The CNCF graduated it. Everyone from startups to Fortune 500 companies runs their workloads on K8s.

Except here's the thing nobody wants to admit: Kubernetes might be winning on paper, but it's bleeding users in practice. And I'm not talking about small shops jumping ship. I'm seeing serious engineering teams at growing companies actively choosing *anything but Kubernetes*.

The Problem Nobody Talks About

Let's start with the elephant in the room. Kubernetes is mind-numbingly complex. Not "oh I need to learn a few new concepts" complex. More like "I need to hire a dedicated platform team just to keep this thing running" complex.

As one Reddit engineer put it in a brutally honest thread, "The mesh isn't a nice to have. It's survival. Once you've got 50+ microservices calling each other, you either add a service mesh or you build a bad one by accident."

The average multicloud environment now spans 12 different platforms and services, and organizations are using an average of 10 different observability or monitoring tools just to keep tabs on everything. That's not simplicity. That's a full-time job disguised as infrastructure.

What's Actually Happening Out There

Walk into any tech company these days and ask the platform team how they feel about Kubernetes. You'll get one of two responses: either nervous laughter or a thousand-yard stare usually reserved for war veterans.

Mean Time To Recovery (MTTR) has been increasing for three years straight. In 2021, 47% of teams could recover from incidents in under an hour. By 2024? That number dropped to just 18%. Meanwhile, 82% of teams now take over an hour to recover. Let that sink in. Our infrastructure is supposed to be getting better, but recovery times are getting worse.

The Alternatives That Are Actually Gaining Ground

Here's where it gets interesting. While everyone argues about whether Kubernetes is too complex, smart teams are quietly building production systems with alternatives that just… work.

Docker Swarm is having a renaissance. I know, I know. Everyone declared it dead years ago. But guess what? For teams that already know Docker (which is basically everyone), Swarm offers orchestration without the PhD. It's simple, it integrates

natively with your existing Docker workflow, and setup takes minutes instead of days.

A platform engineer I know switched their mid-size SaaS company from Kubernetes to Swarm and cut their operational overhead by 60%. Their deployment process went from a multi-day affair requiring specialized knowledge to something any developer on the team could handle.

HashiCorp Nomad is the one that's really making waves. It's lightweight, it handles both containerized and non-containerized workloads, and here's the kicker: you don't need a dedicated team to manage it. It's a single binary. That's it. No complex cluster management, no intricate networking rules, no three-day training sessions.

Teams using Nomad report they can onboard new engineers in hours instead of weeks. When your infrastructure is understandable, you ship faster. Wild concept, right?

Managed services like AWS ECS, Azure Container Instances, and Google Cloud Run are eating Kubernetes' lunch in the "I just want to run containers" segment. These services abstract away the orchestration entirely. You point them at a container, they run it, they scale it, they monitor it. Done.

Sure, you get some vendor lock-in. But you also get to actually build features instead of debugging DNS issues in your service mesh at 2 AM.

Where Kubernetes Actually Makes Sense

Look, I'm not here to bash Kubernetes entirely. It has legitimate use cases. If you're Netflix-scale with hundreds of microservices across multiple data centers and regions, yeah, you probably need something as powerful as Kubernetes.

If you need fine-grained traffic control, complex routing rules, multi-tenancy isolation, or you're running a platform that serves other platforms, Kubernetes makes sense. The complexity is justified when you need that level of control.

But here's the reality check: most companies aren't Netflix. Most companies are trying to run 5–20 microservices reliably and wondering why they need to learn about Custom Resource Definitions and Operators to deploy a web app.

The Cost Nobody Calculates

Everyone talks about Kubernetes being “free” because it’s open source. But free software plus expensive engineers equals very not free infrastructure.

A mid-level platform engineer who actually knows Kubernetes well costs $150–200k annually. You need at least two (because on-call rotations). That’s $300–400k per year before you factor in the opportunity cost of what they could be building instead of wrestling with your cluster.

Meanwhile, ECS on AWS or Cloud Run on GCP might cost you a bit more in compute, but you’re not paying six figures for someone to babysit your orchestration layer.

What the Data Shows

Recent surveys show that only 10% of organizations have fully implemented observability for their Kubernetes clusters. That means 90% of companies running Kubernetes don’t fully understand what’s happening in their own infrastructure.

Complexity remains the number one concern among practitioners, cited by 39% of respondents, followed closely by signal-to-noise challenges at 38%. Alert fatigue was identified as the single biggest blocker to faster incident response.

When 39% of your users say your biggest problem is complexity, that’s not a minor issue. That’s an existential crisis.

The Simpler Future

Here’s what I think is actually happening. We’re hitting an inflection point where teams are realizing that “industry standard” doesn’t mean “right for you.”

Companies are increasingly choosing the right tool for their actual needs rather than defaulting to whatever technology has the most GitHub stars. Docker Swarm, Nomad, managed container services, even serverless platforms, they all have legitimate places in the modern infrastructure stack.

The “best” orchestrator is the one that lets your team ship features without needing a dedicated platform team. Sometimes that’s Kubernetes. Often, it’s not.

The Bottom Line

Kubernetes isn’t dying. Let’s be clear about that. But its dominance is definitely cracking. The assumption that “you must use Kubernetes” is finally being challenged, and that’s healthy for the entire ecosystem.

If you're starting a new project today, really ask yourself: do I need Kubernetes, or do I just think I need it because everyone else uses it? Can Docker Swarm or Nomad solve my actual problems with 90% less complexity? Will a managed service let me focus on my product instead of my infrastructure?

The container orchestration war isn't over. Kubernetes won the first battle, but the war for simplicity and developer experience is just beginning. And right now, Kubernetes is losing that one.

What's your experience been? Are you running Kubernetes and loving it, or are you one of those teams quietly moving to simpler alternatives? Let's hear the real stories.

Containers Docker Kubernetes Programming Cloud Computing

Follow

Written by Sohail Saifi

12.8K followers · 2.5K following

Software Development Engineer | Writing the future — code, AI & more.

Responses (4)

qiulang

What are your thoughts?